Final Submission Report on Cypress

Introduction

This project focused on implementing an end-to-end testing framework using Cypress for the Mini Shop application. The primary objective of the activity was to develop a modular Cypress automation framework using custom commands, fixture files, environment variables, session handling, API interception, flaky test debugging techniques, and automated Mochawesome HTML reporting.

Through this activity, practical understanding was gained regarding how real-world automation frameworks are structured and how Cypress commands are used to create stable, reusable, scalable, and maintainable automated test cases.

Custom Commands Implementation

During the implementation process, reusable custom commands were created inside the `commands.js` file to simplify repeated operations and improve maintainability of the Cypress framework. These commands helped reduce code duplication and made the test cases cleaner and easier to manage.

The following implementation was added inside the `commands.js` file:

```
Cypress.Commands.add("login",(email,password)=>{
    cy.visit("/login");
    cy.get("#email").type(email);
    cy.get("#password").type(password);
    cy.contains("button","Login").click();
});


Cypress.Commands.add("addProducts",(ProductDetails)=>{
    cy.visit("/dashboard");
    cy.contains("button","Go to Add Product").click().wait(500);
    cy.url().should("include","add-product");
    cy.get("h1").should("contain.text","Add New Product");
```

```
    cy.get("form").then(($form)=>{
        cy.wrap($form).find('[class="form-group"]').should("have.length",3);
    });

    cy.get("#name").type(ProductDetails.productName);
    cy.get("#price").type(ProductDetails.productPrice);
    cy.get("#category").type(ProductDetails.productCategory);

    cy.contains("button","Add Product").click();

    cy.on("window : alert",(alertMessage)=>{
        expect(alertMessage).should("contain.text","Product added successfully!");
        return true;
    });
});

Cypress.Commands.add("addNewProducts",(productName, productPrice,
productCategory)=>{
    cy.visit("/dashboard");

    cy.contains("button","Go to Add Product").click().wait(500);

    cy.url().should("include","add-product");
```

```
    cy.get("h1").should("contain.text","Add New Product");

    cy.get("form").then(($form)=>{
        cy.wrap($form).find('[class="form-group"]').should("have.length",3);
    });

    cy.get("#name").type(productName);
    cy.get("#price").type(productPrice);
    cy.get("#category").type(productCategory);

    cy.contains("button","Add Product").click();

    cy.on("window : alert",(alertMessage)=>{
        expect(alertMessage).should("contain.text","Product added successfully!");
        return true;
    });
});

Cypress.Commands.add("cancelAddNewProductsAction",(productName,
productPrice, productCategory)=>{
    cy.visit("/dashboard");

    cy.contains("button","Go to Add Product").click().wait(500);

    cy.url().should("include","add-product");
```

```
    cy.get("h1").should("contain.text","Add New Product");


    cy.get("form").then(($form)=>{
        cy.wrap($form).find('[class="form-group"]').should("have.length",3);
    });


    cy.get("#name").type(productName);
    cy.get("#price").type(productPrice);
    cy.get("#category").type(productCategory);


    cy.contains("button","Cancel").click();
});
```

The `login` custom command was implemented to automate user authentication by entering email and password values dynamically. This command was reused across multiple test cases to avoid repetitive login code.

The `addProducts` custom command was implemented to add products dynamically using object-based product data. The command validated the product form structure, entered product information, and verified successful product creation through alert validation.

The `addNewProducts` command was implemented to support fixture-based testing by dynamically adding multiple products using fixture file data. This improved scalability and enabled reusable test execution for multiple product datasets.

The `cancelAddNewProductsAction` command was implemented to validate the cancel functionality during product creation. This helped verify navigation flow and cancel operations without submitting product data.

The implementation of reusable custom commands improved readability, maintainability, scalability, and overall framework organization.

Test Using Custom Commands

The following test case was implemented using custom commands:

```
describe("Cypress Mini Shop UI Testing - using custom commands", () => {

    beforeEach(() => {

        cy.fixture("userDetails").then((user) => {

            cy.login(user.email,user.password).wait(500);

        });

        cy.then(()=>{

            cy.url().should("include","dashboard");

        });

    });

    it("Add New Product",()=>{

        cy.addProducts({

            "productName" : "Mobile",
```

```
        "productPrice" : "$1500",
        "productCategory" : "Electronics"

    });

  });

});
```

This implementation successfully validated the reusable login functionality and dynamic product addition functionality using custom Cypress commands.

Use of Fixtures and Environment Variables

Fixture files and environment variables were implemented to manage reusable test data and secure configuration values.

The following implementation was created:

```
describe("Cypress Mini Shop UI Testing - using fixtures and env variables", () => {

  beforeEach(()=>{

    cy.login(Cypress.env("email"), Cypress.env("password"));

    cy.url().should("include","dashboard");

  });
```

```
    it("Add New Products using fixture files",()=>{

        cy.fixture("ProductDetails").each((details)=>{

            cy.addNewProducts(
                details.ProductName,
                details.ProductPrice,
                details.ProductCategory
            );

        });

    });

});
```

Environment variables were used to store login credentials securely, while fixture files were used to store reusable product information.

This implementation improved security, maintainability, flexibility, and data-driven testing capability.

Implementation of cy.session() and cy.intercept()

The `cy.session()` command was implemented to cache authenticated user sessions and avoid repeated login execution before every test.

The `cy.intercept()` command was implemented to monitor API requests and validate backend responses.

The following implementation was created:

describe("Cypress Mini Shop UI Testing - using session", () => {

```
beforeEach(() => {

    cy.session("Store Login Session", () => {

        cy.login(Cypress.env("email"), Cypress.env("password"));

    });

    cy.intercept("GET", "**/dashboard*").as("dashboard");

    cy.visit("/dashboard");

    cy.wait("@dashboard").then((interception) => {

        expect(interception.response.statusCode).to.eql(200);

    });

    cy.url().should("include", "dashboard");

});

it("Add New Products", () => {
```

```
    cy.fixture("ProductDetails").each((details) => {

        cy.addNewProducts(
            details.ProductName,
            details.ProductPrice,
            details.ProductCategory
        );

    });

});

it("Cancel Add New Products Action", () => {

    cy.fixture("ProductDetails").each((details) => {

        cy.cancelAddNewProductsAction(
            details.ProductName,
            details.ProductPrice,
            details.ProductCategory
        );

    });

});
```

```
});
```

During execution, the login session was successfully cached and reused across multiple test cases. The intercepted dashboard API request was validated successfully using the response status code.

This implementation improved synchronization, reduced repeated login execution, and enhanced test stability.

Flaky Test Scenario and Resolution

During the activity, flaky test handling techniques were implemented using `cy.pause()` and `cy.debug()`.

The following implementation was created:

```
describe("Cypress Mini Shop UI Testing - handling flaky test", () => {

  beforeEach(() => {

    cy.session("Store Login Session", () => {

      cy.login(Cypress.env("email"), Cypress.env("password"));

    });

    cy.intercept("GET", "**/dashboard*").as("dashboard");

    cy.visit("/dashboard");

    cy.wait("@dashboard").then((interception) => {
```

```
        expect(interception.response.statusCode).to.eql(200);

    });

    cy.url().should("include", "dashboard");

});

it("Add New Products", () => {

    cy.fixture("ProductDetails").each((details) => {

        cy.addNewProducts(
            details.ProductName,
            details.ProductPrice,
            details.ProductCategory
        );

    });

    cy.pause();

    cy.url().should("include","dashboard");
```

```
        cy.get("h1")
          .debug()
          .should("contain.text","Product Dashboard");


    });


});
```

Initially, synchronization delays and timing issues caused unstable test execution. To investigate the issue, `cy.pause()` was used to temporarily stop execution and inspect the application state manually.

The `cy.debug()` command was used to inspect DOM elements and runtime behavior inside the browser console.

After implementing proper synchronization using `cy.wait("@dashboard")`, the flaky behavior was resolved successfully.

This activity helped improve understanding of synchronization handling, debugging techniques, and stable automation practices.

Modular Test Structure

The project was organized using modular test files for different functionalities such as:

- custom commands,
- fixtures and environment variables,
- session handling,
- API interception,
- and flaky test handling.

This modular framework structure improved readability, maintainability, scalability, and organization of the automation suite.

Mochawesome HTML Report

Mochawesome reporting was configured to generate detailed HTML test execution reports after successful test execution.

The generated reports included:

- execution summaries,
- passed and failed test statistics,
- screenshots,
- logs,
- and debugging information.

The Mochawesome report improved result visibility and simplified failure analysis during automation execution.

C:\Users\crayo\Music\coursera\Cypress Mini\Shop\mochawesome-report\mochawesome.html

Outcome of the Activity

The activity was completed successfully by implementing:

- reusable custom commands,
- fixture-based testing,
- environment variable handling,
- session management using `cy.session()`,
- API interception using `cy.intercept()`,
- flaky test handling using `cy.pause()` and `cy.debug()`,
- modular test organization,
- and Mochawesome HTML reporting.

The final framework produced stable, reusable, scalable, and maintainable automated test execution.

Conclusion

Through this activity, practical experience was gained in implementing advanced Cypress automation testing concepts for real-world web applications. The project improved understanding of API synchronization, reusable framework design, session handling, debugging techniques, fixture-based testing, and automated reporting.

The use of custom commands, fixtures, environment variables, `cy.session()`, `cy.intercept()`, flaky test handling techniques, and Mochawesome reporting significantly improved the quality and maintainability of the automation framework.

Overall, this project strengthened practical knowledge and confidence in using Cypress for end-to-end automation testing.